AppName — Full Development Roadmap

> **From zero to production-grade general-purpose real-time simulation software.**
> Every step = software runs. Every step = new capability added.
> Team: Krunal · Veer · Aarav · Sujal *(Project name TBD — placeholder used throughout)*

Team Split

Person	Domain	What They Own
Krunal	Backend	Serial communication, data pipeline, filtering, WebSocket, simulation engine logic
Veer	Firmware + Backend	ESP32 firmware, hardware communication, sensor math, serial protocol
Aarav	Frontend — 3D	3D canvas, rendering, animation, visualization drivers
Sujal	Frontend — UI	All PyQt6 panels, layouts, dialogs, status bars, theming, UI state management

> **Documentation rule:** Whoever builds a feature documents it. Every person writes docstrings for every function they write, every step.

How to Read This Roadmap

```
STEP N — Title
├── Goal          → What the software can do after this step that it couldn't before
├── Learn         → What each person must understand before building their part
├── Parallel Work → What each person builds
├── Commit        → GitHub commit + tag checkpoint
└── Done When     → How you confirm this step is complete
```

Three phases:

- **PHASE 1 (Steps 1–7):** MVP. 5 days. College submission. Arm simulation. Live demo.
- **PHASE 2 (Steps 8–13):** Refactor + General-Purpose Engine. Code becomes clean, extensible, not arm-locked.
- **PHASE 3 (Steps 14–30):** Production. Full features, polish, packaging, release.

PHASE 1 — MVP: College Submission

> **Timeline: 5 days Scope: Robotic arm simulation driven by live glove sensor data. GUI: PyQt6 desktop app 3D: Matplotlib 3D embedded in PyQt6 (fast to build, good enough for demo) The architecture here is intentionally arm-specific. We clean that up in Phase 2.**

STEP 1 — Hardware Verified + Project Skeleton

Goal

ESP32 prints sensor values to the PC terminal. Python project folder exists with the right structure. Git repo is live on GitHub. Every teammate has it cloned.

What To Learn

Krunal:

- What is a serial port? How USB Serial communication works (data sent as bits at a fixed baud rate)
- What `pyserial` is — Python's library to read a serial port
- How to create a Python project with `poetry init` and install packages with `poetry add`
- What `if __name__ == "__main__"` means and why it matters

Veer:

- What is I2C? (2-wire protocol: clock + data — allows multiple sensors on the same 2 wires)
- Arduino IDE setup for ESP32 (board manager URL, how to install, how to flash)
- What `Wire.h` is and how I2C works in Arduino
- What the MPU-6050 gives you: raw accelerometer (g-force) and gyroscope (degrees/second)
- What `Serial.println()` does and how serial output reaches the PC
- What `millis()` returns in Arduino

Aarav:

- What a Python virtual environment is and why it matters (dependency isolation)
- Git fundamentals: repository, branch, commit, push, pull, clone
- What `QApplication` and `QMainWindow` are in PyQt6 — the two mandatory objects every PyQt6 app starts with

Sujal:

- What a Python virtual environment is and why it matters
- Git fundamentals: same as above
- What a Widget is in PyQt6 (every visible element — button, label, window — is a widget)
- What `QHBoxLayout` and `QVBoxLayout` do (arrange widgets side-by-side or top-to-bottom)

Parallel Work

Person	Task
Veer	Wire MPU-6050 to ESP32: 3.3V→VCC, GND→GND, GPIO21→SDA, GPIO22→SCL. Flash the MPU6050 example sketch. Confirm raw accelerometer + gyro values printing in Arduino Serial Monitor at 115200 baud. Move sensor → numbers change.
Krunal	Create GitHub repo. Set up folder structure (see Architecture doc). `poetry init` inside `backend/`. Install `pyserial`. Write a 15-line script `serial_test.py` that opens the serial port and prints every line it receives. Confirm it prints what Veer's ESP32 is sending.
Aarav	Clone repo. Create `frontend/` folder. Write `app.py` — creates `QApplication`, instantiates `MainWindow`, calls `show()` and `app.exec()`. Window can be blank. Confirm it opens without errors.
Sujal	Clone repo. Write `frontend/main_window.py` — a `MainWindow` class inheriting `QMainWindow`. Use `QHBoxLayout` to place two empty `QWidget` placeholders side by side (left panel and right panel). Window should open showing two visible empty areas.

Commit Checkpoint

```
[firmware] MPU6050 raw values printing over serial at 115200
[backend]  serial_test.py connects and prints incoming data
[frontend] app.py opens MainWindow without errors
[frontend] MainWindow has two-panel skeleton layout
```

Done When

- Veer moves the sensor → numbers change in Arduino Serial Monitor
- Krunal's script prints the same numbers in Python terminal
- `poetry run python app.py` opens a window with two side-by-side panels

STEP 2 — GUI Shell + Dark Theme + JSON Firmware Output

Goal

The app has a real dark-themed window. Left panel has a connection interface (port selector, Connect button, status label). Right panel is still empty. Clicking Connect doesn't connect to anything real yet — just changes the status label. Veer's ESP32 now outputs JSON instead of raw numbers.

What To Learn

Veer:

- Why JSON is better than raw text for sensor output (structured, self-describing, easy to parse on any platform)
- How to output a JSON string from ESP32: `Serial.println("{\"r\":12.3,\"p\":-5.1,\"y\":88.2}")`

Krunal:

- Write `backend/config.py` with default constants: port name, baud rate, WebSocket port
- `pyserial`'s `serial.tools.list_ports.comports()` — returns a list of available serial ports on the machine

Aarav:

- What a Qt stylesheet is: `widget.setStyleSheet("background: #111111; color: #eeeeee;")`
- How to apply a stylesheet to the whole application: `app.setStyleSheet(...)`
- `QSplitter` — a resizable divider between two panels

Sujal:

- `QLabel` — displays text
- `QLineEdit` — a single-line text input
- `QPushButton` — a clickable button
- `QComboBox` — a dropdown selector
- `QGroupBox` — a labelled container that visually groups related controls
- Connecting a button click to a Python function: `btn.clicked.connect(my_function)`

Parallel Work

Person	Task
Veer	Update firmware to output agreed JSON: `{"t":1234,"r":12.3,"p":-5.1,"y":88.2}`. Confirm in Serial Monitor.
Krunal	Write `backend/config.py` with defaults. Write `list_available_ports()` using pyserial — returns list of port name strings.

Person	Task
Aarav	Apply a dark stylesheet app-wide in `app.py`. Style both panels with visible background colours. Add the app name as a styled `QLabel` header.
Sujal	Build `frontend/panels/connection_panel.py` — a `QGroupBox` "Connection" with: port `QComboBox` (populated using Krunal's `list_available_ports()`), Connect `QPushButton`, status `QLabel` showing "Disconnected" in red. Button click changes label to "Connected" in green (fake). Embed in left panel of `MainWindow`.

Commit Checkpoint

```
[firmware] JSON output format implemented and confirmed
[backend]  config.py with defaults, list_available_ports() working
[frontend] dark theme applied application-wide
[frontend] connection panel with port selector, button, status label
```

Done When

App opens dark-themed. Connection panel visible on left. Clicking Connect changes label colour. Veer's ESP32 outputs JSON.

STEP 3 — Real Serial Connection + Live Data Readout

Goal

Clicking Connect actually opens the serial port. Real Roll/Pitch/Yaw values from the ESP32 update on screen as live numbers. No 3D yet. Just numbers.

What To Learn

Krunal:

- What is a thread and why we need one here (serial reading in the main thread freezes the GUI — must run in background)
- `QThread` in PyQt6 — the correct way to run background work alongside a GUI
- PyQt6 signals and slots: the only safe way to pass data from a background thread to the GUI (`pyqtSignal`)
- Never touch a GUI widget directly from a background thread — always use signals
- `json.loads()` — parses a JSON string into a Python dict
- `try/except` around serial reads — handles unexpected disconnects without crashing

Veer:

- Ensure stable 50Hz+ JSON output — no garbled lines, no partial lines
- `Serial.println()` always terminates with a newline — use this, not `Serial.print()`
- Add sensor init error: if MPU-6050 fails, output `{"error":"sensor_init_failed"}`

Aarav:

- `FigureCanvasQTAgg` — how to embed a Matplotlib figure as a PyQt6 widget
- How to create a dark-background 3D `Axes3D` figure (dark `fig.patch`, dark `ax` background)
- Embed an empty styled 3D canvas in the right panel — nothing drawn yet, just the dark axes

Sujal:

- How to update a `QLabel`'s text dynamically: `.setText("new text")`
- `QFrame` — a styled container widget
- Build a "Data Readout" panel: three large bold labels for Roll, Pitch, Yaw with degree symbols, and a packets/sec counter
- Expose `update_values(roll, pitch, yaw)` method on this panel

Parallel Work

Person	Task
Veer	Tune to stable 50Hz+. Add error JSON on sensor init failure. Document serial protocol (baud, JSON fields, rate) in `firmware/README.md`.
Krunal	Write `backend/serial_worker.py` — `QThread` subclass that opens port, reads lines in a loop, parses JSON, emits `data_received(dict)` signal. Emits `error(str)` signal on failures. Handles clean shutdown when stopped.
Aarav	Write `frontend/panels/arm_canvas.py` — `FigureCanvasQTAgg` subclass with a dark styled 3D `Axes3D`. No arm drawn yet. Embed in right panel of `MainWindow`.
Sujal	Write `frontend/panels/data_panel.py` — `QFrame` with large Roll/Pitch/Yaw labels and packets/sec counter. `update_values()` method updates all labels. Embed below the connection panel on the left. Connect to `SerialWorker.data_received` signal in `MainWindow`.

Commit Checkpoint

```
[backend]  SerialWorker QThread: reads, parses JSON, emits data_received
signal
[backend]  error signal on port failure or malformed JSON
[frontend] dark 3D canvas embedded in right panel
[frontend] data_panel live Roll/Pitch/Yaw update from signal
```

Done When

Connect → numbers update in real time. Packets/sec shows ~50. Unplugging USB doesn't crash the app. Dark 3D canvas visible on the right (empty but styled).

STEP 4 — Static 3D Arm

Goal

The right panel shows a 3D robotic arm — static, not moving yet, but correctly structured. Base → shoulder → upper arm → elbow → lower arm → wrist → gripper. Styled to look like real simulation software.

What To Learn

Krunal:

- What a joint and link are in a robotic arm (joint = rotation point, link = rigid segment between joints)
- 3D coordinate system: X, Y, Z axes and the right-hand rule
- What a rotation matrix does (a 3×3 matrix that rotates a 3D point around an axis)
- `numpy` arrays for 3D points: `np.array([x, y, z])`
- `np.dot()` — matrix multiplication (applying a rotation to a point)
- Write `backend/kinematics.py`: `compute_arm_positions(roll, pitch, yaw)` returns a list of 6 3D points. Start with all angles = 0 → arm is straight upright.

Veer:

- No firmware change this step
- Verify Krunal's kinematics output is geometrically correct (draw it on paper — does the math give the right shape for zero angles?)

Aarav:

- `ax.plot()` — draws a line between two 3D points in Matplotlib
- `ax.scatter()` — draws a dot/sphere at a 3D point
- `ArmCanvas.draw_arm(positions)` — takes 6 points, draws arm segments as thick lines, joint positions as coloured dots
- Styling: cyan/white line segments, coloured joint spheres (base=grey, shoulder=blue, elbow=orange, wrist=green, gripper=red)

Sujal:

- `QSizePolicy` — how widgets grow/shrink when the window is resized

- `setMinimumSize()` — prevent the window from becoming too small to use
- `QStatusBar` — the strip at the bottom of a `QMainWindow` for status messages
- Finalise layout: left panel fixed width, right panel expands with window resize. Status bar shows "Disconnected".

Parallel Work

Person	Task
Veer	Verify kinematics math with Krunal. Write full `firmware/README.md`: component list, ASCII wiring diagram, IDE setup, library install, flash steps.
Krunal	Write `backend/kinematics.py`. Confirm zero angles gives geometrically correct straight arm with Veer.
Aarav	Implement `ArmCanvas.draw_arm(positions)`. Call it once with Krunal's zero-angle output so a static styled arm appears.
Sujal	Finalise layout: fixed left column, expanding right column, resize behaviour correct. Add `QStatusBar` showing connection state. Add window icon.

Commit Checkpoint

```
[backend]  kinematics.py compute_arm_positions() returns correct 3D joint
positions
[frontend] ArmCanvas.draw_arm() renders styled static 3D arm
[frontend] layout finalized with correct resize behaviour
[frontend] status bar showing connection state
```

Done When

App shows styled 3D arm (5 joints, styled segments) on the right. Data panel on the left. Status bar at bottom. Arm is static but looks correct.

STEP 5 — Live Arm Movement (Full End-to-End)

Goal

The arm moves in real time from live sensor data. Move the glove → arm mirrors it on screen. The entire pipeline works: glove → ESP32 → Python → GUI → 3D arm.

What To Learn

Krunal:

- Why raw IMU data is noisy: gyroscope drifts over time, accelerometer is noisy during fast motion
- Complementary filter: `angle = alpha * (angle + gyro_rate * dt) + (1 - alpha) * accel_angle`
- What each term does: gyro term tracks fast changes, accel term corrects long-term drift
- `time.monotonic()` — reliable elapsed time measurement (better than `time.time()` for intervals)
- Write `backend/filter.py` implementing this formula

Veer:

- Increase output to 100Hz
- Add raw gyro fields to JSON (needed for the complementary filter): `{"t":1234,"r":12.3,"p":-5.1,"y":88.2,"gx":0.1,"gy":-0.2,"gz":0.05}`

Aarav:

- How to redraw a Matplotlib 3D figure efficiently: `ax.cla()` then redraw (clear and replot)
- `canvas.draw_idle()` — schedules a redraw without blocking the event loop
- `QTimer` — fire a function at a fixed interval. Use one at 30fps to trigger redraws (decouples render rate from sensor rate — prevents GUI overload at 100Hz)
- Add `update_arm(positions)` method to `ArmCanvas`

Sujal:

- What sensor calibration means (zeroing the neutral position)
- Add a "Calibrate" button to the connection panel: on click, records current roll/pitch/yaw as the zero offset. All future readings subtract this offset.
- Button disabled until connected, enabled once connected

Parallel Work

Person	Task
Veer	Update firmware: 100Hz, add raw gyro fields to JSON. Confirm in Serial Monitor.
Krunal	Write `filter.py`. Wire into `SerialWorker`: raw JSON → parse → filter → compute kinematics → emit `arm_updated(list)` signal with 6 joint positions.
Aarav	Add `update_arm(positions)` to `ArmCanvas`. Add `QTimer` at 30fps calling `canvas.draw_idle()`. Connect `arm_updated` signal to `update_arm`.
Sujal	Implement calibration: on click, read current angles, store as offset in `MainWindow`, pass to `SerialWorker` for subtraction from all future readings.

Commit Checkpoint

```
[firmware] 100Hz output with raw gyro fields in JSON
[backend]  filter.py complementary filter applied to live data
[backend]  arm_updated signal emits filtered 3D joint positions
[frontend] ArmCanvas updates at 30fps from live sensor data
[frontend] calibration button zeros neutral position
```

Done When

Wear glove → run app → Connect → Calibrate → move hand → arm mirrors movement in real time. This is the MVP core feature.

STEP 6 — Connection Stability + Error States

Goal

Every failure is handled gracefully. Wrong port → clear error message. Sensor disconnects mid-demo → auto-reconnect attempt. No crashes. No frozen UI. Demo-safe.

What To Learn

Krunal:

- `pyserial` exceptions: `SerialException`, `SerialTimeoutException` — what causes each
- Connection state machine concept: DISCONNECTED → CONNECTING → CONNECTED → ERROR → DISCONNECTED
- Auto-retry: attempt reconnect every 3 seconds without blocking the UI (`QTimer` in main thread)
- Emit `status_changed(state: str, message: str)` signal from `SerialWorker` on every state transition

Veer:

- Test every failure by hand: unplug mid-session, replug, enter wrong port, send malformed data from Serial Monitor
- Write `docs/test_checklist.md` — every failure scenario and expected behaviour

Aarav:

- Text overlay on a Matplotlib figure: `ax.text2D(x, y, "text", transform=ax.transAxes)`
- Show a "Waiting for connection..." overlay on the 3D canvas when not connected. Hide it when data flows.

Sujal:

- `QMessageBox.warning()` — modal error alert dialog
- Reflect connection state in UI: button greyed while connecting, port dropdown disabled while connected, status label and status bar update on every state change
- `QStatusBar.showMessage(text, timeout_ms)` — show temporary messages

Parallel Work

Person	Task
Veer	Run through every failure mode. Document pass/fail in `docs/test_checklist.md`.
Krunal	Implement full connection state machine in `SerialWorker`. Auto-retry on disconnect. `status_changed` signal on every transition.
Aarav	Add "no signal" text overlay to `ArmCanvas`. Visible when state ≠ CONNECTED. Hidden when data arrives.
Sujal	Wire `status_changed` signal across all UI elements: status label, status bar, button states, port dropdown enabled/disabled. Show `QMessageBox` on ERROR state.

Commit Checkpoint

```
[backend]  connection state machine with auto-reconnect every 3s
[backend]  status_changed signal on every state transition
[frontend] no-signal overlay on arm canvas when disconnected
[frontend] all UI controls reflect connection state correctly
```

Done When

All scenarios in `test_checklist.md` pass. Unplug during demo → overlay appears, reconnect attempted. Replug → reconnects automatically. No crashes anywhere.

STEP 7 — MVP Polish + Demo Preparation

Goal

The software is ready to demo in front of evaluators. A replay backup exists for hardware failures. GitHub is clean and tagged.

What To Learn

Krunal:

- `argparse` — Python's standard library for CLI arguments (`--port COM3` , `--replay file.csv` , `--record file.csv`)
- `csv.writer` / `csv.DictWriter` — saving incoming sensor data to a file with timestamps
- How to replay a CSV file as if it were live serial data (use timestamps to reproduce original timing)

Veer:

- Record a 60-second demo session to CSV as the hardware backup
- Confirm the final firmware build on demo hardware
- Prepare spare jumper wires

Aarav:

- `QDialog` — how to build a simple "About" dialog
- Final camera angle: set a good default 3D view angle for the demo position
- Screenshot the running app for the README

Sujal:

- Window title bar: `"[AppName] v1.0.0-mvp – Real-Time Robotic Arm Simulation"`
- `QMenuBar` with a Help menu: About, Keyboard Shortcuts
- Keyboard shortcuts: `Ctrl+K` = Calibrate, `Escape` = Disconnect, `Ctrl+R` = toggle Replay mode

> 📝 **College Report:** Each person writes their own section. Krunal → Backend. Veer → Hardware. Aarav → Visualization. Sujal → UI/Integration. Each person knows their part best.

Parallel Work

Person	Task
Veer	Record backup CSV. Stress-test hardware for 30 minutes. Prepare demo kit (ESP32 + glove + spare wires).
Krunal	Add `--replay` and `--record` CLI args. Replay mode feeds saved CSV data through the same pipeline as live serial.
Aarav	Final arm visual polish. Good default camera angle. About dialog. App screenshot.
Sujal	Window title, menu bar, keyboard shortcuts. Write final `README.md` : screenshot, description, install steps, hardware setup link, how to run.

Commit Checkpoint

```
[backend]  --replay and --record CLI arguments working
[firmware] final build confirmed on demo hardware
[frontend] About dialog, menu bar, keyboard shortcuts
[frontend] final visual polish, good default camera angle
[docs]     README complete with screenshot
git tag v1.0.0-mvp
```

Done When

Two working demo paths: (1) live hardware, (2) `--replay backup.csv`. GitHub is clean. Tag `v1.0.0-mvp` on main. Report written.

COLLEGE SUBMISSION COMPLETE

> Take a break. Submit. Then return and continue from Step 8.

PHASE 2 — Refactor + General-Purpose Simulation Engine

> **This phase does two things: 1. Clean up the MVP codebase into a proper maintainable architecture (MVC). 2. Deliberately break the arm-specific coupling so the software can simulate anything. No flashy new features. But after this phase, every future feature is faster and safer to build.**

STEP 8 — MVC Architecture Refactor

Goal

Every module has exactly one job. GUI knows nothing about serial ports. Serial reader knows nothing about the GUI. Data flows through a clean central pipeline (MVC pattern). This is the foundation everything else builds on.

What To Learn

Krunal:

- MVC pattern: Model = data and logic, View = what the user sees, Controller = the bridge between them
- `dataclasses` in Python: define typed `SensorReading` and `ArmState` data structures cleanly

- Abstract Base Classes (`ABC`): define a `DataSource` interface that both `SerialSource` and `ReplaySource` implement
- Dependency injection: pass objects in as arguments — never import shared globals

Veer:

- Git branching for large refactors: `refactor/architecture` branch — only merge into main when everything runs correctly

Aarav:

- Python packages and relative imports: `from .arm_canvas import ArmCanvas`
- How to split one large file into multiple small focused files without breaking imports
- `__init__.py` — what it does and when you need it

Sujal:

- Create `frontend/theme.py` : every hardcoded colour string, font size, and spacing value in the entire codebase moves here
- Semantic versioning: MAJOR.MINOR.PATCH — what each number means and when to bump each
- Start `CHANGELOG.md` — one entry per tag going forward

New Folder Structure

```
[AppName]/
├── app.py                          ← entry point only (10 lines max)
├── core/
│   ├── models/
│   │   ├── sensor_reading.py       ← SensorReading dataclass
│   │   └── simulation_state.py     ← SimulationState dataclass (generic,
not arm-specific yet)
│   ├── sources/
│   │   ├── base.py                 ← DataSource ABC
│   │   ├── serial_source.py
│   │   └── replay_source.py
│   └── processing/
│       ├── filter.py
│       └── pipeline.py             ← orchestrates: source → filter →
simulation
├── simulations/
│   └── arm/
│       └── kinematics.py           ← arm-specific logic (isolated here,
not in core)
└── ui/
    ├── main_window.py
    ├── theme.py
    └── panels/
```

```
        ├── connection_panel.py
        ├── data_panel.py
        └── arm_canvas.py
```

Notice: arm-specific code is already isolated in `simulations/arm/`. This is intentional — it sets up the next step.

Commit Checkpoint

```
[refactor] restructure into core/ + simulations/ + ui/ layout
[refactor] SensorReading and SimulationState dataclasses
[refactor] DataSource ABC with Serial + Replay implementations
[refactor] pipeline.py orchestrates the data flow
[refactor] theme.py centralises all visual constants
git tag v1.1.0
```

Done When

`poetry run python app.py` still works identically to v1.0.0-mvp. New folder structure in place. All arm code is inside `simulations/arm/` — not scattered through `core/`.

STEP 9 — General-Purpose Simulation Engine

Goal

This is the most important architectural step in the entire roadmap. The software stops being "an arm simulator that happens to receive sensor data" and becomes "a real-time simulation engine that can load any simulation driver." The robotic arm becomes the first simulation driver. Future simulations (humanoid skeleton, drone, robotic leg, anything) are just new drivers dropped in.

Why This Step Exists

Right now the pipeline is:

```
SerialSource → Filter → kinematics.py (arm-specific) → ArmCanvas (arm-specific)
```

After this step it becomes:

```
DataSource → Filter → SimulationEngine → SimulationDriver (arm, or anything else) → SimulationCanvas
```

The engine doesn't know or care what it's simulating. Only the driver does.

What To Learn

Krunal (owns the engine):

- Abstract Base Classes in depth: how to define an interface that forces subclasses to implement specific methods
- What a "driver" pattern is: a standardised interface to a specific implementation (same idea as a printer driver — OS sends standard commands, driver handles the hardware specifics)
- What the engine needs to know: nothing about arm geometry. Only: "here is sensor data, give me back renderable state"
- Define `SimulationDriver` ABC with these methods:
  - `process(sensor_reading: SensorReading) -> SimulationState` — core method
  - `get_name() -> str` — human-readable name of this simulation
  - `get_description() -> str`
  - `reset()` — reset to neutral/zero state
- `SimulationEngine` class: holds a reference to the current driver, calls `process()` on each incoming reading, emits `simulation_updated(SimulationState)` signal
- `ArmDriver` — the existing arm kinematics wrapped in this interface (first concrete implementation)

Veer:

- Understand the new pipeline (important — firmware output format is the first input to this engine)
- No firmware changes needed — the interface to hardware doesn't change, only how data is processed internally

Aarav:

- Define `SimulationCanvas` ABC: a PyQt6 widget that accepts a `SimulationState` and renders it
  - `render(state: SimulationState)` — core method
  - `reset_camera()` — reset viewpoint
  - `get_name() -> str`
- `ArmCanvas` becomes `ArmSimulationCanvas` — the first concrete implementation
- The canvas doesn't know about sensor data at all. It only knows about `SimulationState`.

Sujal:

- Simulation selector in the UI: a `QComboBox` in the connection panel showing available simulation drivers by name
- When a different driver is selected, the active canvas switches accordingly

- For now only "Robotic Arm" is available — but the selector is there, ready for more

What `SimulationState` Looks Like (Generic)

```
@dataclass
class SimulationState:
    driver_name: str          # which driver produced this
    timestamp: int            # milliseconds
    renderable: Any           # driver-specific data passed to the canvas
    metadata: dict            # any extra info (joint angles, confidence,
etc.)
```

The `renderable` field is intentionally untyped — it's whatever the driver and its paired canvas have agreed to exchange. The engine doesn't inspect it.

Parallel Work

Person	Task
Krunal	Write `core/engine/simulation_driver.py` — `SimulationDriver` ABC. Write `core/engine/simulation_engine.py` — `SimulationEngine` class. Wrap existing `kinematics.py` into `simulations/arm/arm_driver.py` implementing `SimulationDriver`. Wire engine into `pipeline.py`.
Veer	Verify end-to-end still works after refactor. Write a second tiny test driver `simulations/debug/raw_driver.py` that just passes raw roll/pitch/yaw through as `renderable` — useful for debugging the pipeline.
Aarav	Write `ui/canvas/simulation_canvas.py` — `SimulationCanvas` ABC. Refactor `ArmCanvas` into `simulations/arm/arm_canvas.py` implementing `SimulationCanvas`. Update `MainWindow` to hold a `SimulationCanvas` reference, not a direct `ArmCanvas`.
Sujal	Add simulation selector `QComboBox` to connection panel. Build `SimulationRegistry` — a dict mapping driver name → (driver class, canvas class). `MainWindow` uses registry to instantiate the right pair when selection changes.

Final Folder Structure After This Step

```
[AppName]/
├── app.py
├── core/
│   ├── models/
│   │   ├── sensor_reading.py
│   │   └── simulation_state.py        ← generic, driver-agnostic
│   ├── sources/
│   │   ├── base.py
│   │   ├── serial_source.py
```

```
│   │   └── replay_source.py
│   ├── engine/
│   │   ├── simulation_driver.py       ← SimulationDriver ABC ⭐
│   │   ├── simulation_engine.py       ← SimulationEngine class ⭐
│   │   └── simulation_registry.py     ← maps names to driver+canvas pairs
⭐
│   └── processing/
│       ├── filter.py
│       └── pipeline.py
├── simulations/
│   ├── arm/
│   │   ├── arm_driver.py              ← implements SimulationDriver ⭐
│   │   └── arm_canvas.py             ← implements SimulationCanvas ⭐
│   └── debug/
│       └── raw_driver.py              ← simple test driver
└── ui/
    ├── main_window.py
    ├── theme.py
    ├── canvas/
    │   └── simulation_canvas.py       ← SimulationCanvas ABC ⭐
    └── panels/
        ├── connection_panel.py        ← now has simulation selector
        ├── data_panel.py
        └── arm_canvas.py             ← now imports from simulations/arm/
```

Commit Checkpoint

```
[engine] SimulationDriver ABC defined
[engine] SimulationEngine class wired into pipeline
[engine] SimulationRegistry maps names to driver+canvas pairs
[simulations] ArmDriver implements SimulationDriver
[ui] SimulationCanvas ABC defined
[ui] ArmCanvas implements SimulationCanvas
[ui] simulation selector in connection panel
git tag v1.2.0
```

Done When

- `poetry run python app.py` still works exactly as before — arm simulation runs correctly
- Selecting "Robotic Arm" in the dropdown loads the arm driver + canvas
- A second developer can add a new simulation by: writing a new `SimulationDriver` subclass, writing a new `SimulationCanvas` subclass, and registering them in `SimulationRegistry` — without touching any existing code
- This is the test: adding a simulation should require zero changes to `core/`

STEP 10 — Automated Testing + CI

Goal

Core logic (filter, kinematics, engine, parser) has automated tests. `pytest` completes in under 10 seconds. GitHub shows a green checkmark badge after every push.

What To Learn

Krunal:

- `pytest` basics: test functions, `assert` statements, running `pytest` from terminal
- `pytest.approx()` — comparing floats with tolerance
- `@pytest.fixture` — shared setup shared across multiple tests
- Test the filter: known input → assert output within expected range
- Test the engine: create a mock driver, feed sensor readings, assert correct `SimulationState` emitted

Veer:

- Write tests for the JSON format contract: valid JSON → correct `SensorReading`, malformed JSON → exception caught, partial line → skipped gracefully
- What "edge case testing" means: test the boundaries, not just the happy path

Aarav:

- (GUI testing is hard at this stage — focus on understanding what the tests cover and running them)

Sujal:

- Write `.github/workflows/test.yml` — GitHub Actions CI config that runs `pytest` on every push to every branch
- Add a test status badge to `README.md` (`![Tests](https://github.com/.../.../actions/workflows/test.yml/badge.svg)`)
- What CI/CD means: Continuous Integration catches bugs before they reach main

Commit Checkpoint

```
[tests] pytest suite: filter, kinematics, engine, serial parser
[tests] edge case tests: malformed JSON, extreme values, zero values
[tests] mock SimulationDriver for engine testing
[ci]    GitHub Actions runs pytest on every push
git tag v1.3.0
```

STEP 11 — Configuration System

Goal

Nothing is hardcoded. Port, baud rate, filter alpha, arm segment lengths, UI colours — all in `config.yaml`. Settings dialog lets users change everything without touching code.

What To Learn

Krunal:

- YAML format: what it is, how it compares to JSON (more readable, supports comments)
- `pydantic` — Python data validation: define a config schema with types and constraints
- `pydantic-settings` — loads YAML config into a pydantic model automatically
- Config file convention: `~/.appname/config.yaml` (user), `config.default.yaml` in repo (defaults)

Veer:

- (No firmware changes — confirm config-driven arm segment lengths still look correct in simulation)

Aarav:

- Update `ArmCanvas` to read arm segment lengths and colours from config instead of hardcoding them

Sujal:

- Build `ui/panels/settings_dialog.py` — a `QDialog` with a form for all config options
- `QFormLayout` — cleanest layout for settings (label + input side by side per row)
- `QDoubleSpinBox` for floats, `QCheckBox` for booleans, `QComboBox` for enums
- Save button writes to `config.yaml`. Cancel discards changes.
- Add "Settings" to the menu bar

Commit Checkpoint

```
[backend]  pydantic config model, loads from config.yaml
[frontend] arm_canvas reads segment lengths + colours from config
[frontend] settings dialog reads and writes config
git tag v1.4.0
```

STEP 12 — Logging System

Goal

The app writes structured log files. Every connection event, error, and anomaly is timestamped and recorded. `print()` is gone from the entire codebase.

What To Learn

Krunal:

- Python `logging` module: `logger = logging.getLogger(__name__)` — one logger per module
- Log levels: DEBUG (everything), INFO (normal events), WARNING (unexpected but not fatal), ERROR (something broke), CRITICAL (app cannot continue)
- `StreamHandler` (terminal output), `RotatingFileHandler` (file, auto-rotates at size limit)
- Replace every `print()` in the backend with the correct `logger.level()` call

Veer:

- (No firmware changes — review what deserves DEBUG vs WARNING in the serial reader)

Aarav:

- Optionally log render frame time at DEBUG level in `ArmCanvas`

Sujal:

- Write `ui/panels/log_panel.py` — a `QPlainTextEdit` widget showing live log output (scrollable, read-only, dark-themed, monospace font)
- Hidden by default, toggle with `Ctrl+L`
- Write a `QtLogHandler` — a `logging.Handler` subclass that emits a PyQt signal so the log panel receives Python log entries in real time

Commit Checkpoint

```
[backend]  structured logging across all modules, RotatingFileHandler
[backend]  all print() statements replaced with log calls
[frontend] log viewer panel hidden by default, Ctrl+L to toggle
git tag v1.5.0
```

PHASE 3 — Core Features

> **The simulation engine is now clean and general-purpose. Every feature added here builds on that foundation. When adding a new simulation later, only a new**

> **driver + canvas is needed — zero core changes.**

STEP 13 — Improved 3D Rendering (Replace Matplotlib → VisPy)

Goal

The 3D canvas is replaced with VisPy — a real OpenGL-based renderer. 60fps smooth animation, proper lighting, depth shading, mouse orbit/zoom. And critically: the `SimulationCanvas` ABC means swapping renderers requires changing only the arm canvas, nothing else.

What To Learn

Aarav (owns this step):

- What is OpenGL? (low-level GPU rendering API — draws triangles extremely fast)
- What is a GPU and why 3D rendering uses it (renders millions of pixels in parallel)
- VisPy library: `vispy.scene`, `visuals.Cylinder`, `visuals.Sphere`, `visuals.Line`
- Embedding VisPy in PyQt6: `SceneCanvas(app='pyqt6')` as a widget
- Scenegraph: a tree of 3D objects where parent transforms apply to all children (this is how joints chain)
- Linear interpolation (lerp): `current + (target - current) * 0.2` — smoothly blends toward target each frame

Sujal:

- Update `theme.py`: VisPy uses `(r, g, b, a)` tuples in range 0–1, not CSS hex strings
- Add toolbar above the 3D canvas: "Reset Camera" button, "Wireframe / Solid" toggle
- `QToolBar` — how to create a toolbar with buttons and separators

Krunal:

- Add VisPy to `pyproject.toml`. Confirm `SimulationState` data format works for the new canvas.

Veer:

- Test with hardware. Confirm arm motion is smooth at 60fps.

Commit Checkpoint

```
[frontend] ArmCanvas replaced with VisPy OpenGL implementation
[frontend] 60fps rendering with mouse orbit/zoom
[frontend] lerp interpolation for smooth arm animation
```

```
[frontend] 3D view toolbar: Reset Camera + Wireframe toggle
git tag v2.0.0
```

STEP 14 — A Second Simulation: Free-Body Rotation Viewer

Goal

Add a second simulation driver: "Free-Body Rotation Viewer" — a single 3D object (a cube or sphere) that rotates freely based on sensor orientation. No arm, no joints. Just raw orientation visualised on a 3D object.

Why this step matters: This is the first proof that the general-purpose engine works. Adding this simulation requires zero changes to `core/`. Only two new files: a driver and a canvas.

What To Learn

Krunal:

- Write `simulations/freeBody/free_body_driver.py` — implements `SimulationDriver`. `process()` returns a `SimulationState` with `renderable = {"quaternion": [...]}` or `{"roll": ..., "pitch": ..., "yaw": ...}`
- Register in `SimulationRegistry`

Veer:

- Confirm sensor data is sufficient for free-body rotation (it is — roll/pitch/yaw directly map to Euler rotation)

Aarav:

- Write `simulations/freeBody/free_body_canvas.py` — implements `SimulationCanvas`. Renders a textured VisPy box that rotates according to the incoming orientation. Add axis arrows (X=red, Y=green, Z=blue) for reference.

Sujal:

- "Free-Body Rotation Viewer" now appears in the simulation selector dropdown
- When selected, canvas switches to the new viewer. Data panel updates label names to match (no "Roll/Pitch/Yaw for joint X" — just "Roll / Pitch / Yaw" for the object).

Commit Checkpoint

```
[simulations] FreeBodyDriver implements SimulationDriver
[simulations] FreeBodyCanvas implements SimulationCanvas
```

```
[simulations] registered in SimulationRegistry
[ui] simulation selector shows both options, switching works live
git tag v2.1.0
```

Done When

You can switch between "Robotic Arm" and "Free-Body Rotation Viewer" in the dropdown while the hardware is connected. Each simulation responds to live sensor data. Zero changes were made to `core/`.

STEP 15 — Data Recording & Playback

Goal

Record button saves all incoming sensor data to a timestamped file. Open Recording replays any saved session with accurate original timing. Timeline scrubber lets you jump to any point.

What To Learn

Krunal:

- `csv.DictWriter` — writing structured rows with headers
- Why timestamps in recordings matter: playback must reproduce original timing, not replay all at once
- Update `ReplaySource` to read timestamps and `time.sleep()` the correct interval between rows

Veer:

- Record reference datasets: slow rotation, fast rotation, full range of motion, figure-8. Store in `recordings/reference/` in the repo. These become standard test inputs for future development.

Aarav:

- "REPLAY" badge overlay on the canvas during playback (so it's obvious this isn't live data)
- Playback speed controls above the canvas: 0.5x / 1x / 2x buttons

Sujal:

- "Recording" panel below the data panel: Record button (turns red when active), Stop button, Open File button
- `QFileDialog.getSaveFileName()` / `getOpenFileName()` — standard file picker dialogs

- `QSlider` timeline scrubber showing playback progress and allowing seeking
- Elapsed time `QLabel` next to the scrubber

Commit Checkpoint

```
[backend]  recording to timestamped CSV
[backend]  replay with accurate original timing
[frontend] recording panel with record/stop/open controls
[frontend] timeline scrubber, speed controls, replay badge
git tag v2.2.0
```

STEP 16 — Gesture Recognition

Goal

Define named gestures ("Fist", "Open Hand", "Point") by performing them and clicking "Save Gesture". Software recognises them in real time and displays the name on screen.

What To Learn

Krunal:

- What a classifier is: takes input data → outputs a category label
- KNN (K-Nearest Neighbors): compare current snapshot to saved examples — nearest match wins. Simple, explainable, no separate training step.
- `scikit-learn`: `KNeighborsClassifier`, `joblib.dump()` / `joblib.load()` for saving/loading the model
- Confidence thresholding: only report a gesture if nearest match is close enough (Euclidean distance below a threshold)

Veer:

- Record 10 samples of each gesture for training data. Test recognition accuracy.

Aarav:

- Gesture name overlay on the canvas: large text showing current detected gesture
- Colour-code overlays by gesture (each gets a distinct accent colour from `theme.py`)

Sujal:

- "Gestures" panel: list of saved gestures with Name, Sample Count, Delete button
- "Add Gesture" dialog: name input, Capture button (saves current sensor snapshot as a sample), sample counter, Save button

Commit Checkpoint

```
[backend]  gesture capture + KNN classifier + confidence threshold
[backend]  real-time gesture recognition signal
[frontend] gesture management panel and add-gesture dialog
[frontend] detected gesture overlay on canvas
git tag v2.3.0
```

STEP 17 — WiFi Mode (Wireless Glove)

Goal

ESP32 connects to WiFi and streams over WebSocket. No USB cable needed. The glove is wireless.

What To Learn

Veer (owns firmware side):

- ESP32 WiFi: `WiFi.h`, `WiFi.begin(ssid, password)`, `WiFi.localIP()`
- ESP32 WebSocket client library (`WebSocketsClient`)
- Architecture: ESP32 is the WebSocket **client**, Python is the **server**
- Reconnection on the ESP32 side when WiFi drops

Krunal:

- Write `WiFiSource` — a `DataSource` that runs a WebSocket server instead of reading serial
- `websockets` Python async server: `websockets.serve(handler, host, port)`
- `qasync` — integrates Python `asyncio` event loop with the Qt event loop
- Auto-detect local IP: display it in the UI so the user knows what to configure on the ESP32

Aarav:

- No canvas changes — data format is identical, source is different

Sujal:

- Connection mode selector: `QComboBox` toggling between "USB Serial" and "WiFi"
- WiFi mode shows: local IP address ESP32 should connect to, connection indicator turning green when first data arrives
- Serial port dropdown hidden in WiFi mode

Commit Checkpoint

```
[firmware] WiFi + WebSocket client mode on ESP32
[backend]  WiFiSource WebSocket server
[backend]  asyncio + Qt event loop via qasync
[frontend] connection mode selector (Serial / WiFi)
git tag v2.4.0
```

STEP 18 — Advanced Sensor Fusion (Madgwick Filter)

Goal

Replace the complementary filter with the Madgwick algorithm. Eliminates yaw drift. Handles fast aggressive motion correctly. Accurate enough for precision applications.

What To Learn

Krunal (owns this step):

- Gimbal lock: why Euler angles (roll/pitch/yaw) fail at certain orientations (when two rotation axes align)
- Quaternions: a 4-number representation of 3D rotation that avoids gimbal lock
- Madgwick filter: fuses gyro + accelerometer data using quaternion math to produce a drift-free orientation
- `ahrs` Python library — has ready-made Madgwick and Mahony implementations
- Converting quaternion output to Euler angles for display, or passing quaternions directly to the simulation driver

Veer:

- Test with hardware: rapid spins, extreme angles. Compare arm behaviour before and after filter change.

Aarav + Sujal:

- No changes — output is still roll/pitch/yaw, just more accurate

Commit Checkpoint

```
[backend] Madgwick filter replacing complementary filter
[backend] quaternion-based orientation tracking internally
git tag v3.0.0
```

STEP 19 — Data Analysis Dashboard

Goal

A second tab shows live charts: roll/pitch/yaw over the last 10 seconds, packet rate, raw vs filtered comparison. Lets you analyse sensor data quality without external tools.

What To Learn

Aarav:

- `pyqtgraph` — fast real-time plotting built for PyQt (much faster than Matplotlib for live charts)
- Ring buffer (circular buffer): fixed-size array that overwrites oldest data — perfect for "last N seconds"
- `pg.PlotWidget` — a drop-in PyQt6 widget for line charts

Sujal:

- `QTabWidget` — adding tabs to a window
- Add "Analytics" tab next to the main "Simulation" tab
- Dashboard layout: Roll chart, Pitch chart, Yaw chart stacked vertically, each showing raw (dim) and filtered (bright) data overlaid

Krunal:

- Expose rolling window buffer from the pipeline to the analytics tab

Commit Checkpoint

```
[backend]  ring buffer exposing rolling window data
[frontend] analytics tab with live pyqtgraph charts
[frontend] raw vs filtered data overlay per axis
git tag v3.1.0
```

STEP 20 — Simulation Driver SDK

Goal

Writing a new simulation driver is formally documented, easy, and doesn't require reading the core codebase. There is a template, a guide, and a working example. Anyone on the team (or a contributor) can add a new simulation in an afternoon.

What To Learn

Krunal:

- Write `simulations/template/` — a minimal driver + canvas template with comments explaining every method
- Write `docs/creating_simulations.md` — step-by-step guide: copy template, implement the two required methods, register in `SimulationRegistry`, test with `--replay`

Veer:

- Build a third simulation using only the template + guide (no help from Krunal): "Gyroscope Visualiser" — a VisPy scene showing the three rotation axes as animated arrows. Tests whether the SDK guide is good enough on its own.

Aarav:

- Write `simulations/template/template_canvas.py` — minimal canvas template with comments

Sujal:

- Add "Add Simulation" developer guide link in the Help menu
- Simulation info panel: when a driver is selected in the dropdown, show its `get_name()` and `get_description()` below the selector

Commit Checkpoint

```
[sdk]      simulation driver + canvas template
[sdk]      creating_simulations.md guide
[simulations] gyroscope visualiser built using only the template guide
git tag v3.2.0
```

STEP 21 — Arm Model Editor

Goal

A built-in editor lets users define their own arm: joint count, segment lengths, joint rotation axes, colours. Three built-in presets: 3-DOF simple, 6-DOF industrial, human arm.

What To Learn

Sujal (owns UI):

- `QTreeWidget` — hierarchical list showing the joint chain
- `QDoubleSpinBox` for editing DH parameters per joint
- Selecting a joint in the tree highlights it in the 3D canvas

- Save/Load arm model as JSON using `QFileDialog`

Krunal:

- Arm model JSON schema: define it, validate with pydantic
- `ArmDriver.load_model(path)` — hot-reload arm model without restarting the app

Aarav:

- Highlight selected joint in the VisPy view (change colour/scale on selection)
- Live preview: arm updates in the canvas as user edits parameters

Commit Checkpoint

```
[backend]  arm model JSON schema + pydantic validation
[backend]  hot-reload arm model without restart
[frontend] arm model editor dialog with tree + parameter fields
[frontend] live preview + selected joint highlighting
git tag v3.3.0
```

STEP 22 — Plugin System

Goal

External Python files dropped into `plugins/` are auto-discovered and loaded. Plugins can hook into the data pipeline and add new simulations or UI panels.

What To Learn

Krunal (owns this step):

- `importlib.util.spec_from_file_location()` — dynamically importing a Python file at runtime
- `GloveArmPlugin` ABC: `on_load(app)`, `on_sensor_data(reading)`, `on_shutdown()` lifecycle methods
- Plugin discovery: `glob.glob("plugins/*.py")` → import each → find `GloveArmPlugin` subclass → instantiate

Sujal:

- Plugin manager dialog: list of discovered plugins, enable/disable toggle per plugin, name + description
- "Plugins" menu item in the menu bar

Commit Checkpoint

```
[backend]  plugin auto-discovery system
[backend]  GloveArmPlugin ABC with lifecycle methods
[frontend] plugin manager dialog
git tag v3.4.0
```

PHASE 4 — Production Grade

STEP 23 — Full UI/UX Redesign

Goal

The application looks like commercial software. Consistent design language, professional typography, smooth transitions, custom-styled widgets throughout.

What To Learn

Sujal (leads this step):

- Qt StyleSheets (QSS) in depth — full CSS-like syntax for every widget type
- `QPropertyAnimation` — smooth animated transitions
- `QIcon` with SVG icons, `QSvgRenderer`
- Typography hierarchy: display font for titles, monospace for data readouts
- Colour theory basics: contrast ratios for WCAG AA accessibility
- **Write `docs/design_system.md` first** — colour palette, type scale, spacing grid. Get team sign-off before touching a single widget.

Aarav:

- Update VisPy environment to match the new design system: lighting, background, arm material colours
- Smooth camera animation on "Reset Camera" click using VisPy's built-in camera interpolation

Commit Checkpoint

```
[frontend] full UI redesign with design system
[frontend] smooth state transitions, consistent icon set
[frontend] VisPy environment matches design system
[docs]     design_system.md
git tag v4.0.0
```

STEP 24 — Security Hardening

Goal

All external input validated before use. No path traversal. No code execution from config or plugins. Logs contain no sensitive data.

What To Learn

Krunal (owns this step):

- Pydantic validation for all external inputs: serial data, config files, recording files, plugin manifests
- Path traversal: never build file paths from raw user input. Use `pathlib.Path.resolve()` and assert the result is inside an expected directory.
- Why `eval()` on user data is always wrong — what to use instead
- Log sanitisation: strip raw sensor data from WARNING+ log levels

Veer:

- Firmware: add a max packet length check. Reject and reset if a line exceeds expected byte count.

Sujal:

- First-launch consent dialog: explain what data is stored locally, confirm nothing is sent externally
- Write `docs/security.md` — threat model for this desktop app (realistic threats only)

Commit Checkpoint

```
[security] pydantic validation on all external inputs
[security] path traversal protection in all file operations
[security] log sanitisation at WARNING+ level
[docs]     threat model document
git tag v4.1.0
```

STEP 25 — Performance Profiling & Optimisation

Goal

< 5% CPU at idle, < 20% CPU during active streaming. Memory stable over hours. Every bottleneck found, fixed, and documented.

What To Learn

Krunal:

- `cProfile` + `snakeviz` — profile the whole app, find slowest functions
- `tracemalloc` — track memory allocations, find leaks
- `line_profiler` — time per line inside the hottest functions
- Signal throttle: batch 100Hz sensor data into 30Hz UI updates with a `QTimer`

Aarav:

- VisPy: `canvas.update()` vs `canvas.draw()` — when to use each
- Reduce geometry complexity for low-end hardware (configurable segment polygon count)

Sujal:

- Add FPS + CPU + memory usage to the status bar using `psutil`
- Performance mode toggle in settings: reduces render quality for slow machines

Everyone:

- Big O notation: O(1), O(n), O(n²) — the standard vocabulary for algorithm efficiency
- Commit `docs/performance_report.md` with profiling results before and after

Commit Checkpoint

```
[backend]  signal throttle at 30fps
[frontend] VisPy rendering optimised, configurable polygon count
[frontend] FPS + CPU + memory in status bar
[docs]     performance_report.md before/after comparison
git tag v4.2.0
```

STEP 26 — Full Documentation Site

Goal

A documentation website a stranger can use to set up and run the software in under 30 minutes. Auto-generated API reference. Every public function has a docstring.

What To Learn

Sujal (leads; everyone contributes their own section):

- `mkdocs` + `mkdocs-material` — converts Markdown to a documentation website (`mkdocs serve` previews locally)
- `mkdocstrings` — auto-generates API reference pages from Python docstrings

- `CONTRIBUTING.md` and `CODE_OF_CONDUCT.md`

Everyone:

- Google docstring format: `Args:`, `Returns:`, `Raises:`, `Example:` — one complete docstring per public function
- Each person writes docs for what they built: Krunal → backend/engine API, Veer → hardware + firmware, Aarav → 3D canvas + simulation driver guide, Sujal → UI panels + theming guide

Commit Checkpoint

```
[docs] mkdocs site with material theme deployed
[docs] auto-generated API reference from docstrings
[docs] every public function has complete docstring
[docs] CONTRIBUTING.md
git tag v4.3.0
```

STEP 27 — Windows .exe Packaging

Goal

One command builds a standalone Windows installer. No Python needed on the user's machine.

What To Learn

Krunal (owns this step):

- `PyInstaller` — bundles Python app + dependencies into a `.exe`
- PyInstaller spec file: controls what's included (VisPy shaders, config defaults, icons, simulation assets)
- Common pitfalls: hidden imports, missing DLLs, data files not found at runtime (all fixable in the spec file)
- `Inno Setup` — creates a proper Windows Setup wizard from the PyInstaller output

Sujal:

- GitHub Actions `release.yml`: on every version tag push (`v*`), automatically build `.exe` and attach to GitHub Release
- Write `INSTALL.md`

Commit Checkpoint

```
[build] PyInstaller spec file producing working .exe
[build] Inno Setup installer script
[ci]    GitHub Actions release workflow builds and uploads .exe on tag push
[docs]  INSTALL.md
git tag v5.0.0
```

STEP 28 — Auto-Updater

Goal

On startup, the software checks GitHub Releases for a newer version. If found, shows a non-intrusive notification with an update option.

What To Learn

Krunal:

- GitHub Releases REST API: `GET /repos/owner/repo/releases/latest`
- `requests` library for HTTP in Python
- `packaging.version.Version` — comparing version strings (`1.2.0 < 1.3.0`)
- Update flow: download new installer → launch it → old app exits → new version installs

Sujal:

- Update notification banner: dismissable, non-intrusive, appears at top of window
- "Check for Updates" in the Help menu (manual check)

Commit Checkpoint

```
[backend]  startup version check via GitHub Releases API
[frontend] update notification banner
[frontend] Help > Check for Updates
git tag v5.1.0
```

STEP 29 — Crash Reporting (Opt-In Only)

Goal

If the app crashes, it offers to send an anonymous crash report. Zero data collected without explicit consent.

What To Learn

Krunal:

- `sys.excepthook` — intercepts all uncaught Python exceptions
- `Sentry` Python SDK: what it captures (traceback, versions), what it never captures (sensor data, file paths, usernames)

Sujal:

- First-launch consent: "Help improve [AppName] — send anonymous crash reports? Yes / No / Ask later"
- Store decision in config. Never ask again after a choice is made.
- Write `docs/privacy_policy.md`

Commit Checkpoint

```
[backend]  Sentry crash reporting, opt-in only
[frontend] first-launch consent dialog
[docs]     privacy_policy.md
git tag v5.2.0
```

STEP 30 — Internationalisation (i18n)

Goal

Every visible UI string is translatable. Adding a new language requires only a translation file — no code changes.

What To Learn

Sujal (owns this step):

- Qt's `QTranslator` and `.ts` / `.qm` translation files
- `pylupdate6` — scans Python source for `self.tr("...")` calls and extracts them into a `.ts` file
- `lrelease` — compiles `.ts` files into binary `.qm` files loaded at runtime
- Wrap every hardcoded UI string in `self.tr(...)` across all UI panels

Aarav:

- (Wrap canvas overlay strings in `tr()`)

Commit Checkpoint

```
[i18n] all UI strings wrapped in tr()
[i18n] English .ts baseline file
[i18n] i18n workflow documented
git tag v5.3.0
```

FINAL: `v1.0.0` — Production Release

```
git tag v1.0.0
```

At this point [AppName] has:

- ✅ General-purpose simulation engine — any rigid-body simulation via driver+canvas pattern
- ✅ Built-in simulations: Robotic Arm, Free-Body Rotation, Gyroscope Visualiser
- ✅ Simulation Driver SDK — documented, templated, contributor-ready
- ✅ WiFi + USB Serial connection modes
- ✅ Madgwick sensor fusion filter (no drift, quaternion-accurate)
- ✅ Gesture recognition (KNN-based, configurable)
- ✅ Inverse kinematics mode for arm simulation
- ✅ Data recording and playback with timeline scrubber
- ✅ Arm model editor with built-in presets
- ✅ Data analysis dashboard (live charts, raw vs filtered)
- ✅ Plugin system for third-party extensions
- ✅ Professional UI with full design system
- ✅ Settings dialog, logging system, configuration file
- ✅ Full documentation site with auto-generated API reference
- ✅ Automated test suite with CI on every push
- ✅ Security hardened and threat-modelled
- ✅ Performance profiled and optimised
- ✅ Standalone Windows .exe installer (auto-built on tag push)
- ✅ Auto-updater
- ✅ Opt-in crash reporting
- ✅ Internationalization-ready

Appendix A — The General-Purpose Engine (How It Works)

After Step 9, this is the complete data flow:

```
Hardware / File
      |
      ▼
DataSource (SerialSource or WiFiSource or ReplaySource)
      |  SensorReading
      ▼
Filter (ComplementaryFilter → Madgwick after Step 18)
      |  filtered SensorReading
      ▼
SimulationEngine
      |  calls active SimulationDriver.process(reading)
      ▼
SimulationDriver  ←── swappable: ArmDriver, FreeBodyDriver, GyroDriver, or
yours
      |  SimulationState(renderable=...)
      ▼
SimulationCanvas  ←── paired with driver: ArmCanvas, FreeBodyCanvas, or
yours
      |
      ▼
VisPy OpenGL (60fps)
      |
      ▼
Screen
```

Adding a new simulation = new driver file + new canvas file + one line in registry. Zero changes to anything else.

Appendix B — Complete Knowledge Map

Krunal (Backend + Engine)

`pyserial`, `websockets`, `asyncio`, `qasync`, `dataclasses`, `pydantic`, `ABC`, `QThread`, `pyqtSignal`, `logging`, `csv`, `argparse`, `cProfile`, `tracemalloc`, `scikit-learn`, `ahrs`, `requests`, `PyInstaller`, `importlib` Concepts: serial communication, WebSocket protocol, MVC, dependency injection, ABC/interface design, complementary filter, Madgwick filter, quaternions, DH kinematics, FABRIK IK, KNN classifier, plugin architecture, security hardening, auto-update pattern, simulation driver pattern

Veer (Firmware + Hardware)

`Wire.h`, `ArduinoJson`, `MPU6050`, `WiFi.h`, `WebSocketsClient` Concepts: I2C protocol, UART/Serial, ESP32 GPIO, ESP32 WiFi, WebSocket client on embedded hardware, MPU-

6050 sensor physics, dual-sensor I2C addressing, baud rate, data framing, firmware reliability, JSON serial protocol design

Aarav (Frontend — 3D)

PyQt6: `FigureCanvasQTAgg`, `QTimer`, signals/slots, `QDialog`, `QTabWidget`, `QToolBar` 3D/Graphics: Matplotlib 3D → VisPy, OpenGL concepts, scenegraph, transformation matrices, lerp interpolation, mouse picking, SimulationCanvas ABC, forward kinematics visualisation Concepts: GPU rendering, frame rate vs data rate, UI/data decoupling, visual feedback design, driver+canvas pattern

Sujal (Frontend — UI)

PyQt6 widgets: `QMainWindow`, `QWidget`, all layout types, `QLabel`, `QLineEdit`, `QPushButton`, `QComboBox`, `QSlider`, `QSpinBox`, `QCheckBox`, `QGroupBox`, `QFrame`, `QSplitter`, `QStatusBar`, `QMenuBar`, `QToolBar`, `QDialog`, `QFileDialog`, `QMessageBox`, `QPlainTextEdit`, `QTreeWidget`, `QTabWidget`, `QPropertyAnimation` Concepts: event loop, signal/slot, UI state machine, QSS stylesheets, design systems, WCAG accessibility, responsive layout, UX for error states, i18n with Qt Linguist

Everyone

Git: branches, commits, tags, pull requests, GitHub Actions, semantic versioning, CHANGELOG Software engineering: MVC, unit testing (pytest), CI/CD, configuration management (pydantic + YAML), structured logging, plugin architecture, performance profiling, security threat modelling, documentation as code (mkdocs)

Appendix C — Git Tag Summary

Tag	Milestone
`v1.0.0-mvp`	College submission — arm simulation MVP
`v1.1.0`	MVC architecture refactor
`v1.2.0`	**General-purpose simulation engine** ⭐
`v1.3.0`	Automated tests + CI
`v1.4.0`	Config system + settings dialog
`v1.5.0`	Logging system + log viewer
`v2.0.0`	VisPy OpenGL rendering
`v2.1.0`	Second simulation: Free-Body Rotation Viewer
`v2.2.0`	Recording and playback
`v2.3.0`	Gesture recognition

Tag	Milestone
`v2.4.0`	WiFi mode
`v3.0.0`	Madgwick filter + quaternions
`v3.1.0`	Analytics dashboard
`v3.2.0`	Simulation Driver SDK
`v3.3.0`	Arm model editor
`v3.4.0`	Plugin system
`v4.0.0`	Full UI/UX redesign
`v4.1.0`	Security hardening
`v4.2.0`	Performance optimisation
`v4.3.0`	Full documentation site
`v5.0.0`	Windows .exe installer
`v5.1.0`	Auto-updater
`v5.2.0`	Crash reporting
`v5.3.0`	Internationalisation
`v1.0.0`	**Production release**

[AppName] — Krunal · Veer · Aarav · Sujal · March 2026